C static code analysis

Unique rules to find Bugs, Vulnerabilities, Security Hotspots, and Code Smells in your C code

1. "memset" should not be used to delete sensitive data
   Vulnerability
2. POSIX functions should not be called with arguments that trigger buffer overflows
   Vulnerability
3. XML parsers should not be vulnerable to XXE attacks
   Vulnerability
4. Function-like macros should not be invoked without all of their arguments
   Bug
5. The address of an automatic object should not be assigned to another object that may persist after the first object has ceased to exist
   Bug
6. "pthread_mutex_t" should be unlocked in the reverse order they were locked
   Bug
7. "pthread_mutex_t" should be properly initialized and destroyed
   Bug
8. "pthread_mutex_t" should not be consecutively locked or unlocked twice
   Bug
9. Functions with "noreturn" attribute should not return
   Bug
10. "memcmp" should only be called with pointers to trivially copyable types with no padding
    Bug
11. Stack allocated memory and non-owned memory should not be freed
    Bug
12. Closed resources should not be accessed
    Bug
13. Dynamically allocated memory should be released
    Bug
14. Freed memory should not be used
    Bug
15. Memory locations should not be released more than once
    Bug

16.

Memory access should be explicitly bounded to prevent buffer overflows
Bug

17.

Printf-style format strings should not lead to unexpected behavior at runtime
Bug

18.

Recursion should not be infinite
Bug

19.

Resources should be closed
Bug

20.

Hard-coded credentials are security-sensitive
Security Hotspot

21.

"goto" should jump to labels declared later in the same function
Code Smell

22.

Only standard forms of the "defined" directive should be used
Code Smell

23.

Switch labels should not be nested inside non-switch blocks
Code Smell

24.

The right-hand operands of && and || should not contain side effects
Code Smell

25.

Digraphs should not be used
Code Smell

26.

Trigraphs should not be used
Code Smell

27.

"case" ranges should cover multiple values
Code Smell

28.

Array indices should be placed between brackets
Code Smell

29.

Redundant pointer operator sequences should be removed
Code Smell

30.

Non-reentrant POSIX functions should be replaced with their reentrant versions
Code Smell

31.

"goto" statements should not be used to jump into blocks
Code Smell

32.

Keywords introduced in later specifications should not be used as identifiers
Code Smell

33.

Switch cases should end with an unconditional "break" statement
Code Smell

34.

"switch" statements should not contain non-case labels
Code Smell

35.

Control should not be transferred into a complex logic block using a "goto" or a "switch" statement
Code Smell

36.

Accessing files should not introduce TOCTOU vulnerabilities
Vulnerability

37.

Cipher algorithms should be robust
Vulnerability

38.

Encryption algorithms should be used with secure mode and padding scheme
Vulnerability

39.

Server hostnames should be verified during SSL/TLS connections
Vulnerability

40.

Server certificates should be verified during SSL/TLS connections
Vulnerability

41.

Cryptographic keys should be robust
Vulnerability

42.

Weak SSL/TLS protocols should not be used
Vulnerability

43.

Insecure functions should not be used
Vulnerability

44.

"scanf()" and "fscanf()" format strings should specify a field width for the "%s" string placeholder
Vulnerability

45.

Function exit paths should have appropriate return values
Bug

46.

"volatile" should not be used to qualify objects for which the meaning is not defined
Bug

47.

Relational and subtraction operators should not be used with pointers to different arrays
Bug

48.

Arguments evaluation order should not be relied on
Bug

49.

Parameter values should be appropriate
Bug

50.

Zero should not be a possible denominator
Bug

51.

Line-splicing should not be used in "//" comments
Bug

52.

Pointers should not be cast to integral types
Bug

53.

"sprintf" should not be used
Security Hotspot

54.

Changing working directories without verifying the success is security-sensitive
Security Hotspot

55.

Using "tmpnam", "tmpnam_s" or "tmpnam_r" is security-sensitive
Security Hotspot

56.

Changing directories improperly when using "chroot" is security-sensitive
Security Hotspot

57.

Using publicly writable directories is security-sensitive
Security Hotspot

58.

Using clear-text protocols is security-sensitive
Security Hotspot

59.

Expanding archive files without controlling resource consumption is security-sensitive
Security Hotspot

60.

Using weak hashing algorithms is security-sensitive
Security Hotspot

61.

Using pseudorandom number generators (PRNGs) is security-sensitive
Security Hotspot

62.

"#undef" should be used with caution
Code Smell

63.

Function names should be used either as a call with a parameter list or with the "&" operator
Code Smell

64.

Functions should not be defined with a variable number of arguments
Code Smell

65.

A cast shall not remove any const or volatile qualification from the type of a pointer or reference
Code Smell

66.

Object and function types should be explicitly stated in their declarations and definitions
Code Smell

67.

Functions should be declared explicitly
Code Smell

68.

Appropriate arguments should be passed to UNIX/POSIX functions
Code Smell

69.

Appropriate arguments should be passed to stream functions
Code Smell

70.

Blocking functions should not be called inside critical sections
Code Smell

71.

Return value of "setuid" family of functions should always be checked
Code Smell

72.

Size of variable length arrays should be positive
Code Smell

73.

Argument of "printf" should be a format string
Code Smell

74.

"mktemp" family of functions templates should have at least six trailing "X"s
Code Smell

75.

Logical operators should not be confused with bitwise operators
Code Smell

76.

Header guards should be followed by according "#define" macro
Code Smell

77.

A conditionally executed single line should be denoted by indentation
Code Smell

78.

Conditionals should start on new lines
Code Smell

79.

Cognitive Complexity of functions should not be too high
Code Smell

80.

Control characters should not be used in literals
Code Smell

81.

"restrict" should not be used
Code Smell

82.

"static" should not be used for the size of an array parameter
Code Smell

83.

The sign of an unsigned variable should not be tested
Code Smell

84.

Pre-defined macros should not be defined, redefined or undefined
Code Smell

85.

Control flow statements "if", "for", "while", "switch" and "try" should not be nested too deeply
Code Smell

86.

"switch" statements should have "default" clauses
Code Smell

87.

Methods should not be empty
Code Smell

88.

Account validity should be verified when authenticating users with PAM
Vulnerability

89.

Lines starting with "#" should contain valid preprocessing directives
Bug

90.

"#include" directives should be followed by either <filename> or "filename" sequences
Bug

91.

Non-standard characters should not occur in header file names in "#include" directives
Bug

92.

Non-empty statements should change control flow or have at least one side-effect
Bug

93.

Unary minus should not be applied to an unsigned expression
Bug

94.

Objects with integer type should not be converted to objects with pointer type
Bug

95.

Variables should be initialized before use
Bug

96.

String literals with different prefixes should not be concatenated
Bug

97.

Only escape sequences defined in the ISO C standard should be used
Bug

98.

"#pragma pack" should be used correctly
Bug

99.

Enums should be consistent with the bit fields they initialize

Bug

100.

Array values should not be replaced unconditionally
Bug

101.

Integral operations should not overflow
Bug

102.

"case" ranges should not be empty
Bug

103.

All branches in a conditional structure should not have exactly the same implementation
Bug

104.

Declaration specifiers should not be redundant
Bug

105.

Unary prefix operators should not be repeated
Bug

106.

"=+" should not be used instead of "+="
Bug

107.

Values of different "enum" types should not be compared
Bug

108.

Conditionally executed code should be reachable
Bug

109.

Null pointers should not be dereferenced
Bug

110.

Single-bit named bit fields should not be of a signed type
Bug

111.

Values should not be uselessly incremented
Bug

112.

Related "if/else if" statements should not have the same condition
Bug

113.

Identical expressions should not be used on both sides of a binary operator
Bug

114.

All code should be reachable
Bug

115.

Loops with at most one iteration should be refactored
Bug

116.

Variables should not be self-assigned

Bug

117.

Setting capabilities is security-sensitive
Security Hotspot

118.

Using "strncpy" or "wcsncpy" is security-sensitive
Security Hotspot

119.

Using "strncat" or "wcsncat" is security-sensitive
Security Hotspot

120.

Using "strcat" or "wcscat" is security-sensitive
Security Hotspot

121.

Using "strlen" or "wcslen" is security-sensitive
Security Hotspot

122.

Using "strcpy" or "wcscpy" is security-sensitive
Security Hotspot

123.

Setting loose POSIX file permissions is security-sensitive
Security Hotspot

124.

#include directives in a file should only be preceded by other preprocessor directives or comments
Code Smell

125.

Loops should not have more than one "break" or "goto" statement
Code Smell

126.

Unused type declarations should be removed
Code Smell

127.

Comma operator should not be used
Code Smell

128.

"bool" expressions should not be used as operands to built-in operators other than =, &&, ||, !, ==, !=, unary &, and the conditional operator
Code Smell

129.

"enum" members other than the first one should not be explicitly initialized unless all members are explicitly initialized
Code Smell

130.

If a function has internal linkage then all re-declarations shall include the static storage class specifer
Code Smell

131.

Functions should not be declared at block scope
Code Smell

132.

Bit fields should be declared with appropriate types

Code Smell

133.

Size of bit fields should not exceed the size of their types
Code Smell

134.

GNU attributes should be used correctly
Code Smell

135.

Unevaluated operands should not have side effects
Code Smell

136.

Size argument of memory functions should be consistent
Code Smell

137.

Implicit casts should not lower precision
Code Smell

138.

Appropriate size arguments should be passed to "strncat" and "strlcpy"
Code Smell

139.

Keywords shall not be used as macros identifiers
Code Smell

140.

Dereferenced null pointers should not be bound to references
Code Smell

141.

"else" statements should be clearly matched with an "if"
Code Smell

142.

Include directives should not rely on non-portable search strategy
Code Smell

143.

"#include" paths should be portable
Code Smell

144.

"#import" should not be used
Code Smell

145.

Atomic types should be used instead of "volatile" types
Code Smell

146.

Methods returns should not be invariant
Code Smell

147.

Printf-style format strings should be used correctly
Code Smell

148.

Conditional operators should not be nested
Code Smell

149.

Multiline blocks should be enclosed in curly braces

Code Smell

150.

Increment should not be used to set boolean variables to 'true'
Code Smell

151.

Boolean expressions should not be gratuitous
Code Smell

152.

Parameters should be passed in the correct order
Code Smell

153.

Obsolete POSIX functions should not be used
Code Smell

154.

Two branches in a conditional structure should not have exactly the same implementation
Code Smell

155.

Unused assignments should be removed
Code Smell

156.

Structures should not have too many fields
Code Smell

157.

"switch" statements should not have too many "case" clauses
Code Smell

158.

Sections of code should not be commented out
Code Smell

159.

Deprecated K&R syntax should not be used for function definition
Code Smell

160.

Unused function parameters should be removed
Code Smell

161.

Unused functions and methods should be removed
Code Smell

162.

Track uses of "FIXME" tags
Code Smell

163.

Deprecated attributes should include explanations
Code Smell

164.

Assignments should not be made from within sub-expressions
Code Smell

165.

Variables should not be shadowed
Code Smell

166.

Redundant pairs of parentheses should be removed
Code Smell

167.

Nested blocks of code should not be left empty
Code Smell

168.

Functions should not have too many parameters
Code Smell

169.

Collapsible "if" statements should be merged
Code Smell

170.

Unused labels should be removed
Code Smell

171.

The "sizeof" and "alignof" operator should not be used with operands of a "void" type
Bug

172.

"nonnull" pointers should not be set to null
Bug

173.

"for" loop counters should not have essentially floating type
Bug

174.

Line continuation characters '\' should not be followed by trailing whitespace
Bug

175.

Using hardcoded IP addresses is security-sensitive
Security Hotspot

176.

Pointer and reference parameters should be "const" if the corresponding object is not modified
Code Smell

177.

The three expressions of a "for" statement should only be concerned with loop control
Code Smell

178.

Literal suffix "L" for long integers shall be upper case
Code Smell

179.

Multicharacter literals should not be used
Code Smell

180.

Loop variables should be declared in the minimal possible scope
Code Smell

181.

Macros should not be used as replacement to "typdef" and "using"
Code Smell

182.

"^" should not be confused with exponentiation
Code Smell

183.

Pointer and reference local variables should be "const" if the corresponding object is not modified
Code Smell

184.

Format strings should comply with ISO standards
Code Smell

185.

Functions which do not return should be declared as "noreturn"
Code Smell

186.

Macros should not be redefined
Code Smell

187.

"#include_next" should not be used
Code Smell

188.

String literals should not be concatenated implicitly
Code Smell

189.

Types and variables should be declared in separate statements
Code Smell

190.

Jump statements should not be redundant
Code Smell

191.

Empty "case" clauses that fall through to the "default" should be omitted
Code Smell

192.

Forward declarations should not be redundant
Code Smell

193.

Declarations should not be empty
Code Smell

194.

Redundant casts should not be used
Code Smell

195.

Code annotated as deprecated should not be used
Code Smell

196.

"#pragma warning (default: ...)" should not be used
Code Smell

197.

Init-declarator-lists and member-declarator-lists should consist of single init-declarators and member-declarators respectively
Code Smell

198.

Unused local variables should be removed
Code Smell

199.

"switch" statements should have at least 3 "case" clauses
Code Smell

200.

A "while" loop should be used instead of a "for" loop
Code Smell

201.

Nested code blocks should not be used
Code Smell

202.

Empty statements should be removed
Code Smell

203.

"/*" and "//" should not be used within comments
Code Smell

204.

Track uses of "TODO" tags
Code Smell

205.

Deprecated code should be removed
Code Smell

206.

Reserved identifiers and functions in the C standard library should not be defined or declared
Code Smell

207.

Bit fields should not be used
Code Smell

208.

Track lack of copyright and license headers
Code Smell

209.

Octal values should not be used
Code Smell

210.

"abort", "exit", "getenv" and "system" from <stdlib.h> should not be used
Bug

211.

"atof", "atoi" and "atol" from <stdlib.h> should not be used
Bug

212.

"<signal.h>" should not be used
Bug

213.

Dynamic heap memory allocation should not be used
Bug

214.

"<time.h>" should not be used
Code Smell

215.

"<stdio.h>" should not be used in production code
Code Smell

216.

"offsetof" macro from <stddef.h> should not be used
Code Smell

217.

"errno" should not be used
Code Smell

218.

"setjmp" and "longjmp" should not be used
Code Smell

219.

Function-like macros should not be used
Code Smell

220.

Macros should not be #define'd or #undef'd within a block
Code Smell

221.

Unions should not be used
Code Smell

222.

Object declarations should contain no more than 2 levels of pointer indirection
Code Smell

223.

Functions without parameters should be declared with parameter type "void"
Code Smell

224.

Recursion should not be used
Code Smell

225.

Constants of unsigned type should have a "U" suffix
Code Smell

226.

Octal and hexadecimal escape sequences should be terminated
Code Smell

227.

Flexible array members should not be declared
Code Smell

228.

Preprocessor directives should not be indented
Code Smell

229.

"switch" statements should not be nested
Code Smell

230.

Cyclomatic Complexity of functions should not be too high
Code Smell

231.

"if ... else if" constructs should end with "else" clauses
Code Smell

232.

"typedef" should be used for function pointers
Code Smell

233.

Control structures should use curly braces
Code Smell

234.

Expressions should not be too complex
Code Smell

235.

Macros used in preprocessor directives should be defined before use
Bug

236.

The number of arguments passed to a function should match the number of parameters
Bug

237.

Evaluation of the operand to the sizeof operator shall not contain side effects
Bug

238.

Bitwise operators should not be applied to signed operands
Bug

239.

Boolean operations should not have numeric operands, and vice versa
Bug

240.

Pointer conversions should be restricted to a safe subset
Bug

241.

Function pointers should not be converted to any other type
Bug

242.

Results of ~ and << operations on operands of underlying types unsigned char and unsigned short should immediately be cast to the operand's underlying type
Bug

243.

User-defined types should not be passed as variadic arguments
Bug

244.

The "<stdlib.h>" functions "bsearch" and "qsort" should not be used
Bug

245.

"sizeof(sizeof(...))" should not be used
Bug

246.

Floating point numbers should not be tested for equality
Bug

247.

There shall be at most one occurrence of the # or ## operators in a single macro definition
Code Smell

248.

Parameters in a function prototype should be named
Code Smell

249.

"goto" statement should not be used
Code Smell

250.

Increment (++) and decrement (--) operators should not be used in a method call or mixed with other operators in an expression
Code Smell

251.

"enum" values should not be used as operands to built-in operators other than [ ], =, ==, !=, unary &, and the relational operators <, <=, >, >=
Code Smell

252.

Operands of "&&" and "||" should be primary (C) or postfix (C++) expressions
Code Smell

253.

Limited dependence should be placed on operator precedence
Code Smell

254.

The value of a complex expression should only be cast to a type that is narrower and of the same signedness as the underlying type of the expression
Code Smell

255.

Braces should be used to indicate and match the structure in the non-zero initialization of arrays and structures
Code Smell

256.

Array declarations should include an explicit size specification
Code Smell

257.

"typedef" names should be unique identifiers
Code Smell

258.

Identifiers should not be longer than 31 characters
Code Smell

259.

All uses of the #pragma directive should be documented
Code Smell

260.

Assembly language should be encapsulated and isolated
Code Smell

261.

Functions that are not used in a project should be removed
Code Smell

262.

"switch" statements should cover all cases
Code Smell

263.

Track parsing failures
Code Smell

264.

Files should not be too complex
Code Smell

265.

The ternary operator should not be used
Code Smell

266.

Functions/methods should not have too many lines
Code Smell

267.

Track uses of "NOSONAR" comments
Code Smell

268.

"for" loop stop conditions should be invariant
Code Smell

269.

Statements should be on separate lines
Code Smell

270.

"switch case" clauses should not have too many lines of code
Code Smell

271.

Functions should not contain too many return statements
Code Smell

272.

Magic numbers should not be used
Code Smell

273.

Files should not have too many lines of code
Code Smell

274.

Lines should not be too long
Code Smell

275.

Function parameters' initial values should not be ignored
Bug

276.

Preprocessor operators "#" and "##" should not be used
Code Smell

277.

Structure and union types should be complete at the end of a translation unit
Code Smell

278.

Switch statement conditions should not have essentially boolean type
Code Smell

279.

"continue" should not be used
Code Smell

280.

Tests of non-Boolean values against zero should be explicit
Code Smell

281.

Signed and unsigned types should not be mixed in expressions
Code Smell

282.

typedefs that indicate size and signedness should be used in place of the basic types
Code Smell

283.

Appropriate char types should be used for character and integer values
Code Smell

284.

Source code should only use /* ... */ style comments
Code Smell

285.

GNU extensions should not be used
Code Smell

286.

Methods should not return constants
Code Smell

287.

Label names should comply with a naming convention
Code Smell

288.

Enumeration values should comply with a naming convention
Code Smell

289.

Enumeration names should comply with a naming convention
Code Smell

290.

Comment styles "//" and "/* ... */" should not be mixed within a file
Code Smell

291.

"union" names should comply with a naming convention
Code Smell

292.

Constants should come first in equality tests
Code Smell

293.

Type specifiers should be listed in a standard order
Code Smell

294.

Track "TODO" and "FIXME" comments that do not contain a reference to a person
Code Smell

295.

The prefix increment/decrement form should be used
Code Smell

296.

"struct" names should comply with a naming convention
Code Smell

297.

File names should comply with a naming convention
Code Smell

298.

Macro names should comply with a naming convention
Code Smell

299.

Comments should not be located at the end of lines of code
Code Smell

300.

break statements should not be used except for switch cases
Code Smell

301.

Local variable and function parameter names should comply with a naming convention
Code Smell

302.

Field names should comply with a naming convention
Code Smell

303.

Lines should not end with trailing whitespaces
Code Smell

304.

Files should contain an empty newline at the end
Code Smell

305.

Tabulation characters should not be used
Code Smell

306.

A function should have a single point of exit at the end of the function
Code Smell

307.

Function names should comply with a naming convention
Code Smell

308.

Track comments matching a regular expression
Code Smell

309.

Track instances of the "#error" preprocessor directive being reached
Code Smell